



Chapitre : Embeddings

Du Texte aux Vecteurs : Tokenisation, Encodage et Représentation Sémantique

4IASDR – IA Distribuée et SMA

Pr. Nadia Idrissi Zouggari – Pr. Hasnââ CHAABI

2025 – 2026

Plan du Chapitre



1. Introduction aux Embeddings



2. Tokenisation : du texte brut aux tokens



3. Encodage des tokens (Token IDs)



4. Vecteurs de base (Token Embeddings)



5. Encodage positionnel (Positional Encoding)



6. Self-Attention : Matrices Q, K, V



7. Du Chunk au Vecteur d'Embedding final



8. Modèles d'Embedding et cas d'usage

01

Introduction aux Embeddings

Pourquoi transformer le texte en vecteurs ?

Pourquoi les Embeddings ?

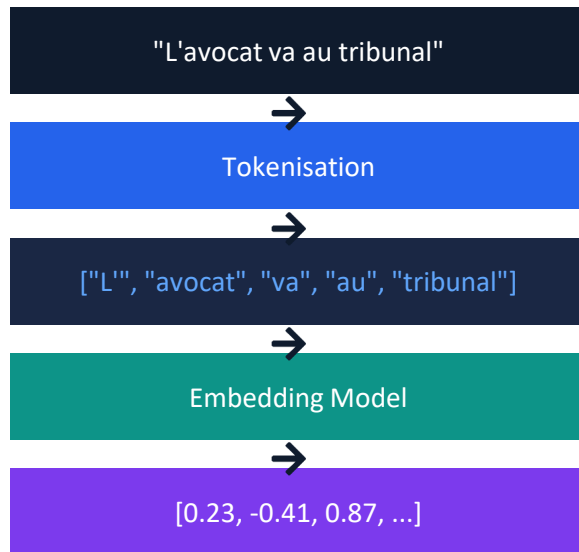
Le problème fondamental

Les modèles de Machine Learning et de Deep Learning ne comprennent que les nombres. Le texte brut ("L'avocat va au tribunal") est incompréhensible pour un réseau de neurones. Il faut donc transformer chaque mot, sous-mot ou token en un vecteur numérique de dimension fixe.

Un embedding est une représentation vectorielle dense d'un élément discret (mot, phrase, document) dans un espace continu de dimension d'où les relations sémantiques sont préservées sous forme de proximités géométriques.

Rappel : Dans le chapitre précédent (Chunking), nous avons découpé les documents en segments. Chaque chunk sera maintenant converti en un vecteur d'embedding pour permettre la recherche sémantique dans un pipeline RAG.

Schéma Conceptuel



Introduction: Espace vectoriel multidimensionnel

GPS	Embedding
Coordonnées GPS [48.85, 2.35] et [40.71, -74.00] 2 coordonnées	[1.85,..., ..., 2.35] et [0.71,..., ..., ..., -7.00] 768 coordonnées
Calculer la distance entre 2 villes	Calculer la distance entre 2 concepts

Recherche classique BM25 (Mots-clés) VS Sémantique :

- Requête utilisateur : "Je n'ai plus d'argent sur mon compte."
- Document A (Maroc Telecom) : "Procédure en cas de solde épuisé."
- Document B (Banque) : "L'argent est le nerf de la guerre."

La recherche **classique** (Ctrl+F) choisira le Document B à cause du mot 'argent'.

L'**Embedding** choisira le Document A car l'angle sémantique (vecteur) de 'solde épuisé' est presque parallèle à 'plus d'argent'.

🔗 Simulateur d'Embeddings & RAG

Découvrez comment l'IA transforme le texte en coordonnées mathématiques (vecteurs) pour rechercher des informations. Ajustez le **"Point Mort"** (Seuil de pertinence) pour voir quels documents sont envoyés au LLM.

🔗 Variables

1. Requête (Query)

Animaux domestiques ▼

"Je cherche des infos sur les animaux qui dorment."

2. Métrique de distance

Simil. Cosinus

Dist. Euclidienne

3. Seuil (Point Mort)

(Minimum requis pour le RAG)

Laxiste (0.0) **0.80** Strict (1.0)

💡 Note pédagogique :

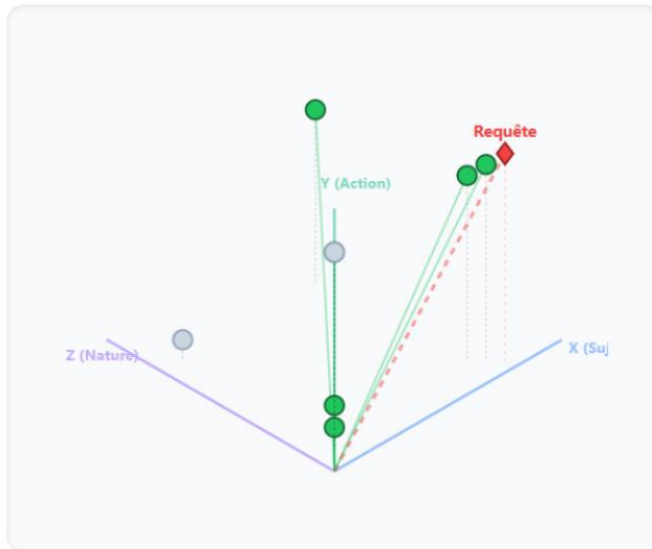
Le Cosinus mesure l'ANGLE entre les vecteurs. Il est excellent pour capturer le thème, peu importe la taille du texte.

🔍 Espace Latent (Embeddings)

2D

3D

Axes : X (Sujet), Y (Action), Z (Nature : Réel vs Jouet). Survolez les points !



◆ Requête (Votre recherche) ● Retenu (Passe le seuil) ● Rejeté (Hors seuil)

📖 Pipeline RAG

1. RÉCUPÉRATION (RETRIEVAL)

- 1.00 Le chat dort confortablement sur le canapé.
- 1.00 Un félin se repose à l'intérieur de la maison.
- 0.84 Un véhicule à moteur accélère sur l'autoroute.
- 0.84 La voiture de sport roule très vite sur le circuit.

2. GÉNÉRATION (PROMPT AUGMENTÉ)

```
# Instruction Système :  
Réponds à la question de l'utilisateur en te  
basant UNIQUEMENT sur le contexte fourni ci-  
dessous.
```

```
# Contexte Récupéré (Injecté par RAG) :
```

- Le chat dort confortablement sur le canapé.
- Un félin se repose à l'intérieur de la maison.
- Un véhicule à moteur accélère sur l'autoroute.
- La voiture de sport roule très vite sur le circuit.
- Plat de pâtes italiennes à la sauce tomate.
- Peluche en forme de chat pour enfant.

Propriétés Clés des Embeddings

Densité

Contrairement au One-Hot Encoding (vecteurs creux de dimension $V \approx 50\,000+$), les embeddings sont des vecteurs denses de dimension $d \approx 384$ à 1536 , où chaque dimension porte de l'information.

Sémantique distribuée

Le sens d'un mot est distribué sur toutes les dimensions. "roi" et "reine" partagent des patterns similaires sur les dimensions liées à la royauté.

Proximité géométrique

Les mots sémantiquement proches ont des vecteurs proches : $\cos(\text{"avocat"}, \text{"juge"}) \approx 0.89$, tandis que $\cos(\text{"avocat"}, \text{"voiture"}) \approx 0.12$.

Opérations algébriques

Les relations sémantiques se traduisent en opérations vectorielles : $\text{vec}(\text{"roi"}) - \text{vec}(\text{"homme"}) + \text{vec}(\text{"femme"}) \approx \text{vec}(\text{"reine"})$.

02

Tokenisation

Du texte brut aux tokens : la première étape du pipeline

Qu'est-ce que la Tokenisation ?

La tokenisation est le processus de découpage d'un texte brut en unités élémentaires appelées tokens. Ces tokens peuvent être des mots entiers, des sous-mots (subwords), ou même des caractères individuels selon l'algorithme choisi. C'est la toute première étape avant de pouvoir générer un embedding : le modèle doit savoir quelles sont les unités à représenter.

Chaque **algorithme de tokenisation** possède un **vocabulaire fixe** (vocab) appris lors de l'entraînement. Les mots fréquents restent entiers, les mots rares sont décomposés en sous-unités connues.

Exemple concret : Phrase = "L'avocat va au tribunal"

Word-level	["L'", "avocat", "va", "au", "tribunal"]	5 tokens
BPE (GPT)	["L", "'", "av", "ocat", "va", "au", "tribunal"]	7 tokens
WordPiece (BERT)	["L", "'", "av", "##ocat", "va", "au", "tribunal"]	7 tokens

Algorithmes de Tokenisation : Comparaison

Critère	BPE	WordPiece	SentencePiece
Principe	Fusionne les paires de bytes les plus fréquentes	Fusionne les paires maximisant la vraisemblance	Traite le texte comme flux de bytes (language-agnostic)
Préfixe	Aucun (continuation implicite)	## pour continuation	— pour début de mot
Modèles	GPT-2, GPT-3, GPT-4, LLaMA	BERT, DistilBERT	T5, ALBERT, XLNet
Vocab typique	50 257 (GPT-2)	30 522 (BERT)	32 000 (T5)
Multilingue	Limité (vocab anglophone)	Moyen (mBERT)	Excellent (byte-level natif)

03

Encodage des Tokens

Attribution d'un identifiant numérique unique à chaque token

Du Token au Token ID

Après la tokenisation, chaque token reçoit un identifiant entier unique (Token ID) issu d'un vocabulaire prédéfini. Ce vocabulaire est une table de correspondance (lookup table) **construite lors de l'entraînement du tokenizer**. Chaque modèle possède son propre vocabulaire.

Exemple : Phrase = "L'avocat va au tribunal"

Tokens	[CLS]	L'	avocat	va	au	tribunal	[SEP]
	→	→	→	→	→	→	→
IDs	101	512	8347	1021	673	29145	102

Résultat : `input_ids = [101, 512, 8347, 1021, 673, 29145, 102]`

04

Vecteurs de Base

Token Embeddings : la représentation vectorielle initiale de chaque token

Token Embedding : Lookup dans la Matrice E

Le modèle possède une matrice d'embedding **E** de dimension $V \times d$, où V est la taille du vocabulaire et d la dimension de l'embedding. Chaque ligne correspond au vecteur de base d'un token. L'opération est un simple lookup : on sélectionne la ligne correspondant au Token ID.

Exemple avec $V = 30\,522$ (BERT) et $d = 4$ (simplifié pour illustration) :

Matrice E ($V \times d$)

ID 0	0.12	-0.34	0.56	0.01
...	...	...	...	...
ID 512 (L')	0.12	-0.34	0.56	0.01
...	...	...	...	...
ID 8347 (avocat)	0.45	0.72	-0.33	0.61
...	...	...	...	...

Résultat du Lookup

Token "L" (ID 512)

[0.12, -0.34, 0.56, 0.01]

Token "avocat" (ID 8347)

[0.45, 0.72, -0.33, 0.61]

Token "tribunal" (ID 29145)

[0.67, 0.88, -0.15, 0.53]

05

Encodage Positionnel

Injecter la notion d'ordre des tokens dans les vecteurs

Pourquoi l'Encodage Positionnel ?

Le Transformer traite tous les tokens en parallèle (contrairement aux RNN qui les traitent séquentiellement). Sans encodage positionnel, le modèle ne peut pas distinguer "L'avocat va au tribunal" de "Au tribunal va l'avocat". On ajoute donc un vecteur de position $PE(pos)$ à chaque token embedding.

L'embedding enrichi devient : $x = \text{Token_Embedding} + \text{Positional_Encoding}$

Formules de l'Encodage Positionnel Sinusoïdal (Vaswani et al., 2017)

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d)})$$

$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d)})$$

Exemple numérique (d=4) : Pour $pos=1$: $PE(1,0) = \sin(1/10000^0) = \sin(1) \approx 0.84$; $PE(1,1) = \cos(1/10000^0) = \cos(1) \approx 0.54$; $PE(1,2) = \sin(1/100) \approx 0.01$; $PE(1,3) = \cos(1/100) \approx 1.00 \rightarrow PE(1) = [0.84, 0.54, 0.01, 1.00]$

Calcul Détaillé : Token Embedding + Position

Token	Position	Token Emb.	Pos. Enc.	=	Embedding Enrichi (x)
L'	0	[0.12, -0.34, 0.56, 0.01]	[0.00, 1.00, 0.00, 1.00]	+	[0.12, 0.66, 0.56, 1.01]
avocat	1	[0.45, 0.72, -0.33, 0.61]	[0.84, 0.54, 0.01, 1.00]	+	[1.29, 1.26, -0.32, 1.61]
tribunal	4	[0.67, 0.88, -0.15, 0.53]	[-0.76, -0.65, 0.02, 1.00]	+	[-0.09, 0.23, -0.13, 1.53]

Détail du calcul pour pos=4 ("tribunal"), d=4

$$PE(4, 0) = \sin(4 / 10000^{(0/4)}) = \sin(4 / 1) = \sin(4) \approx -0.76$$

$$PE(4, 1) = \cos(4 / 10000^{(0/4)}) = \cos(4 / 1) = \cos(4) \approx -0.65$$

$$PE(4, 2) = \sin(4 / 10000^{(2/4)}) = \sin(4 / 100) = \sin(0.04) \approx 0.04$$

$$PE(4, 3) = \cos(4 / 10000^{(2/4)}) = \cos(4 / 100) = \cos(0.04) \approx 1.00$$

06

Self-Attention : Q, K, V

Le mécanisme qui permet à chaque token de 'regarder' tous les autres

Pourquoi le Self-Attention ?

Le problème : un mot peut avoir plusieurs sens

Après les étapes précédentes, chaque token possède un vecteur enrichi (embedding + position). Mais ce vecteur est le même quel que soit le contexte. Observons le problème avec un exemple concret :

"L'avocat va au tribunal"

→ avocat = homme de loi

"L'avocat est bien mûr"

→ avocat = fruit

Problème : Le vecteur du mot "avocat" est identique dans les deux phrases ! Le modèle ne sait pas encore si c'est un fruit ou un métier. Il doit regarder les autres mots de la phrase pour le comprendre.

Solution : le Self-Attention

Le mot "avocat" va poser une question à tous les autres mots de la phrase : "Qui peut m'aider à comprendre mon sens ici ?". Le mot "tribunal" va répondre : "Moi ! Je parle de justice." Et le vecteur d'"avocat" va absorber cette information pour devenir contextualisé.

Self-Attention : L'Analogie de la Bibliothèque

Reprenons la phrase : "L'avocat va au tribunal"

Imaginons que chaque mot est un étudiant dans une bibliothèque. Pour comprendre son propre sens, chaque mot joue 3 rôles simultanément :

Q = Query

"Ce que je cherche"

Le mot "avocat" génère une question :

"Est-ce qu'il y a quelqu'un ici qui parle de justice ou de nourriture ?"

L'étudiant cherche un livre

K = Key

"Ce que j'annonce"

Le mot "tribunal" génère une étiquette :

"Moi, mon thème c'est le droit et la justice."

L'étiquette sur la tranche du livre

V = Value

"L'info que je transmets"

La Query d'"avocat" matche la Key de "tribunal"

→ "tribunal" transmet son contenu (sens juridique) à "avocat".

Le contenu du livre ouvert

Résultat : Le vecteur d'"avocat" absorbe le sens de "tribunal" → il devient contextualisé en "homme de loi" !

Mais comment le modèle génère Q, K, V ?

On a compris que "avocat" doit produire une Query et que "tribunal" doit produire une Key et une Value. Mais concrètement, comment le modèle fabrique ces 3 vecteurs à partir du vecteur enrichi x ?

Réponse : via 3 matrices de poids W_Q , W_K et W_V (appprises automatiquement).

Que sont-elles ?

Des matrices de nombres (poids) de taille $d \times d_k$. Ce sont des paramètres du réseau, comme les poids d'une couche Dense/Linear classique.

D'où viennent-elles ?

Initialisées aléatoirement au début, puis optimisées automatiquement durant l'entraînement (backpropagation + descente de gradient). Le modèle apprend quelles transformations produisent les meilleurs Q, K, V.

Pourquoi 3 séparées ?

Si on utilisait le même vecteur x pour les 3 rôles, "avocat" ne pourrait pas distinguer sa question (Q) de son étiquette (K). 3 matrices = 3 points de vue indépendants = 3 rôles bien distincts.

Analogie : Pensez à W_Q , W_K , W_V comme 3 paires de lunettes différentes. En mettant les "lunettes Query", "avocat" voit le monde comme un chercheur. Avec les "lunettes Key", il voit ce qu'il a à offrir aux autres.

$W_Q \times x = \text{Query}$: Comment ça marche ?

C'est la question clé. Le vecteur x d'"avocat" contient tout son sens (verbe/nom, contexte, position...) mais tout est mélangé. La matrice W_Q agit comme un filtre qui extrait uniquement la partie "question" de ce vecteur.

$$x_{\text{avocat}} \text{ (sens global)} \times W_Q \text{ (filtres à requêtes)} = q_{\text{avocat}} \text{ (la question précise)}$$

Chaque colonne de W_Q est un filtre spécialisé :

Colonne 1

Extrait l'info : "Suis-je un sujet ou un objet ?"

Le vecteur x passe à travers ce filtre → score élevé si x a des traits de sujet

Colonne 2

Extrait l'info : "Quel est mon domaine sémantique ?"

Filtre qui détecte si le mot est lié au droit, à la cuisine, à la nature...

Colonne 3

Extrait l'info : "Quel type de complément je cherche ?"

Filtre qui identifie les attentes contextuelles du mot

Résultat : Le vecteur Query q_{avocat} n'est plus le sens global du mot. C'est une version filtrée et spécialisée qui ne contient que sa "signature de requête" — exactement ce dont il a besoin pour interroger les autres mots.

W_K et W_V : les deux autres filtres

On a vu que W_Q extrait la "question" d'un token. Les matrices W_K et W_V font exactement la même opération de filtrage, mais pour extraire des informations différentes :

W_Q → Query

"Filtre à requêtes"

$$x \times W_Q = q$$

Extrait ce que le token cherche comme contexte

q_avocat : "Je cherche un contexte juridique ou alimentaire"

W_K → Key

"Filtre à étiquettes"

$$x \times W_K = k$$

Extrait ce que le token annonce comme thème

k_tribunal : "Mon thème c'est le droit et la justice"

W_V → Value

"Filtre à contenu"

$$x \times W_V = v$$

Extrait l'information utile à transmettre si le matching est bon

v_tribunal : "Voici mon sens juridique détaillé à transmettre"

Point clé : Les 3 matrices W_Q, W_K, W_V sont les mêmes pour tous les tokens. Mais comme chaque token a un vecteur x différent, le résultat du filtrage est unique pour chacun. C'est comme passer des photos différentes à travers le même filtre Instagram — chaque photo donne un résultat différent.

Chaque token génère ses propres Q, K et V

Phrase : "L' avocat va au tribunal" — Chaque token passe par les 3 filtres simultanément :

Token	x (vecteur enrichi)	$q = x \times W_Q$ (Query)	$k = x \times W_K$ (Key)	$v = x \times W_V$ (Value)
L'	x_L'	"Je suis un article..."	"Article, pas de thème"	"Sens : article défini"
avocat	x_avocat	"Justice ou fruit ?"	"Thème : ambigu"	"Sens : profession/fruit"
va	x_va	"Je suis un verbe de mvt..."	"Thème : mouvement"	"Sens : déplacement"
au	x_au	"Je suis une préposition..."	"Thème : liaison"	"Sens : direction"
tribunal	x_tribunal	"Y a-t-il un sujet juridique ?"	"Thème : droit, justice"	"Sens : lieu de justice"

Ce qu'il faut retenir

Chaque token a maintenant 3 vecteurs spécialisés. Le mot "avocat" a sa Query ("justice ou fruit ?"), sa Key ("thème ambigu"), et sa Value ("sens profession/fruit"). Le mot "tribunal" a sa Key ("droit, justice") qui va **matcher** avec la Query d'"avocat". C'est cette interaction Q·K entre tous les tokens qu'on va voir maintenant.

Les 3 Étapes du Self-Attention

Phrase : "L'avocat va au tribunal" — Voyons comment "avocat" se contextualise :

1

Étape 1 Matching Q·K

La Query d'"avocat" est comparée à la Key de chaque mot (produit scalaire). Score élevé avec "tribunal" (contexte juridique compatible), score faible avec "au" (mot neutre). On divise par $\sqrt{d_k}$ pour stabiliser.

2

Étape 2 Softmax

Les scores deviennent des poids entre 0 et 1 (somme = 1). "tribunal" obtient un poids fort (ex: 0.55), "va" et "au" des poids faibles (ex: 0.10, 0.05). C'est la distribution d'attention : "à qui je dois écouter le plus".

3

Étape 3 Somme pondérée

Le nouveau vecteur d'"avocat" = $0.55 \times \text{Value}(\text{tribunal}) + 0.20 \times \text{Value}(\text{L'}) + 0.10 \times \text{Value}(\text{va}) + 0.05 \times \text{Value}(\text{au}) + 0.10 \times \text{Value}(\text{avocat})$. Le sens de "tribunal" domine → "avocat" est maintenant contextualisé en homme de loi !

Formule : $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$

Interactions Q · K : Qui matche avec qui ?

La Query de chaque token est **comparée** à la Key de tous les autres via un produit scalaire (**dot product**). Un score élevé = forte compatibilité, score faible = peu de lien.

Q \ K	k_L'	k_avocat	k_va	k_au	k_tribunal
q_L'	0.12	0.08	0.15	0.10	0.05
q_avocat	0.10	0.25	0.08	0.06	0.85
q_va	0.18	0.12	0.20	0.30	0.15
q_au	0.08	0.05	0.35	0.10	0.22
q_tribunal	0.06	0.78	0.10	0.18	0.30

Lecture

Score élevé
= forte compatibilité

$q_{avocat} \cdot k_{tribunal} = 0.85$

"avocat" cherche du contexte juridique, "tribunal" l'offre !

$q_{tribunal} \cdot k_{avocat} = 0.78$

$q_{avocat} \cdot k_{tribunal} = 0.85$

$q_{tribunal} \cdot k_{avocat} = 0.78$

$q_{va} \cdot k_{au} = 0.30$

"avocat" pose la question → "tribunal" a la réponse (contexte juridique). Score maximal !

"tribunal" aussi cherche du contexte → "avocat" confirme le thème juridique. Interaction réciproque.

"va" et "au" sont liés syntaxiquement ("va au"), score modéré — lien grammatical, pas sémantique.

Somme Pondérée des Values : la Contextualisation

Après le softmax, chaque token sait "à quel % écouter chaque mot". Maintenant, il mélange les Values de tous les mots selon ces pourcentages. Voici le calcul complet pour "avocat" :

Poids d'attention d'"avocat" après softmax :

L'	0.05 (5%)	avocat	0.10 (10%)	va	0.05 (5%)	au	0.05 (5%)	tribunal	0.75 (75%)
----	-----------	--------	------------	----	-----------	----	-----------	----------	------------

Calcul détaillé (dimension 3 simplifiée)

$$0.05 \times v_{L'} = 0.05 \times [0.1, 0.0, 0.2] = [0.005, 0.000, 0.010]$$

$$0.10 \times v_{\text{avocat}} = 0.10 \times [0.5, 0.3, 0.4] = [0.050, 0.030, 0.040]$$

$$0.05 \times v_{\text{va}} = 0.05 \times [0.2, 0.1, 0.0] = [0.010, 0.005, 0.000]$$

$$0.05 \times v_{\text{au}} = 0.05 \times [0.0, 0.1, 0.1] = [0.000, 0.005, 0.005]$$

$$0.75 \times v_{\text{tribunal}} = 0.75 \times [0.8, 0.9, 0.7] = [0.600, 0.675, 0.525]$$

$$\text{nouveau_avocat} = [0.665, 0.715, 0.580]$$

75% du mélange vient de "tribunal" → le nouveau vecteur d'"avocat" est coloré par le sens juridique. Il est contextualisé !

Calcul Matriciel : Génération de Q, K, V

Exemple avec 3 tokens enrichis ($d=4$) et $d_k=3$ (dimension réduite)

X (3×4)

0.23	0.59	0.87	1.15
1.29	1.26	-0.32	1.61
1.02	-0.97	0.31	1.78

W_Q (4×3)

0.1	0.3	-0.2
0.4	-0.1	0.5
-0.3	0.2	0.1
0.2	0.6	-0.4

×

=

Q (3×3)

0.17	0.86	-0.28
0.83	1.18	-0.44
-0.13	1.47	-1.01

Détail Q[0] : $(0.23 \times 0.1 + 0.59 \times 0.4 + 0.87 \times (-0.3) + 1.15 \times 0.2, \quad 0.23 \times 0.3 + 0.59 \times (-0.1) + 0.87 \times 0.2 + 1.15 \times 0.6, \quad \dots)$
= (0.17, 0.86, -0.28)

De même pour K et V :

$K = X \cdot W_K \rightarrow$ Chaque token produit sa "clé" (ce qu'il offre au matching)

$V = X \cdot W_V \rightarrow$ Chaque token produit sa "valeur" (l'information transmise)

W_K et W_V sont des matrices de poids distinctes, apprises durant l'entraînement.

Scores d'Attention et Softmax

Étape 1 : Calcul des scores bruts via $Q \cdot K^T$

Chaque score mesure la compatibilité entre la query d'un token et la key d'un autre.

$Q \cdot K^T / \sqrt{d_k}$ (3×3)

	L'	avocat	tribunal
L'	0.68	0.15	0.08
avocat	0.10	0.25	0.85
tribunal	0.06	0.78	0.30



Après Softmax (par ligne)

	L'	avocat	tribunal
L'	0.52	0.27	0.21
avocat	0.06	0.14	0.80
tribunal	0.08	0.62	0.30

Interprétation

Lecture ligne par ligne : "avocat" accorde 80% d'attention à "tribunal", 14% à lui-même et 6% à "L".

Le softmax normalise chaque ligne pour que la somme fasse 1.0 (distribution de probabilité).

Les scores élevés sur la diagonale indiquent une forte auto-attention (chaque token s'écoute d'abord).

Sortie du Self-Attention : Vecteurs Contextualisés

Étape finale : Multiplication des poids d'attention par V

Chaque token obtient un nouveau vecteur qui est une somme pondérée des valeurs (V) de tous les tokens, selon les poids d'attention calculés. C'est ainsi que le contexte est injecté dans chaque représentation.

Calcul pour le token "avocat" (ligne 2 de la matrice d'attention)

$$\mathbf{z}_{\text{avocat}} = 0.06 \times \mathbf{V}_{\text{L}'} + 0.14 \times \mathbf{V}_{\text{avocat}} + 0.80 \times \mathbf{V}_{\text{tribunal}}$$

Si $\mathbf{V}_{\text{L}'} = [0.10, 0.05, -0.20]$, $\mathbf{V}_{\text{avocat}} = [0.50, 0.30, 0.40]$, $\mathbf{V}_{\text{tribunal}} = [0.80, 0.90, 0.70]$:

$$\begin{aligned}\mathbf{z}_{\text{avocat}} &= 0.06 \times [0.10, 0.05, -0.20] + 0.14 \times [0.50, 0.30, 0.40] + 0.80 \times [0.80, 0.90, 0.70] \\ &= [0.006, 0.003, -0.012] + [0.070, 0.042, 0.056] + [0.640, 0.720, 0.560]\end{aligned}$$

$$\mathbf{z}_{\text{avocat}} = [0.716, 0.765, 0.604]$$

Ce vecteur $\mathbf{z}_{\text{avocat}}$ encode maintenant "avocat" dans le contexte juridique de la phrase !

07

Du Chunk au Vecteur Final

Pipeline complet : du texte brut au vecteur utilisable en RAG

Pipeline Complet : Chunk → Embedding Vector



Résultat final : embedding = [0.312, -0.087, 0.654, ..., 0.221] (1 vecteur de dimension 768 pour BERT, 1536 pour text-embedding-3-small)

Stratégies de Pooling : Agréger les Vecteurs

Après les couches Transformer, on dispose de N vecteurs contextualisés (un par token). Pour obtenir un seul vecteur représentant tout le chunk, il faut agréger ces vecteurs. C'est l'étape de pooling.

[CLS] Pooling

On utilise le vecteur de sortie du token [CLS] comme représentation du chunk entier. C'est la stratégie originale de BERT.

BERT, RoBERTa

Mean Pooling

Moyenne des vecteurs de tous les tokens : $emb = (1/N) \times \sum z_i$. Souvent meilleur que [CLS] pour la recherche sémantique.

Sentence-BERT, all-MiniLM

Max Pooling

On prend la valeur maximale sur chaque dimension parmi tous les tokens. Capture les caractéristiques les plus saillantes.

Certains modèles spécialisés

Weighted Mean

Moyenne pondérée par les poids d'attention de la dernière couche. Donne plus de poids aux tokens importants.

BGE, E5

08

Modèles d'Embedding

Comparaison des modèles et intégration dans un pipeline RAG

Comparaison des Modèles d'Embedding

Le choix du modèle d'embedding dépend du cas d'usage (recherche sémantique, classification, clustering), de la langue cible, et des contraintes de latence et de coût.

Modèle	Dim.	Max Tokens	Type	Usage recommandé
text-embedding-3-small	1536	8191	API (OpenAI)	RAG général, bon ratio coût/perf.
text-embedding-3-large	3072	8191	API (OpenAI)	Haute précision, multilingue
all-MiniLM-L6-v2	384	512	Open Source	Léger, rapide, idéal prototypage
BGE-large-en-v1.5	1024	512	Open Source	SOTA open-source, MTEB leader
voyage-3	1024	32000	API (Voyage)	Long context, RAG sur gros docs
CamemBERT-base	768	512	Open Source	Français natif, NER/Classification

Dimension: Combien de Features nous pourrions avoir

Max Tokens:

Type:

Embedding dans le Pipeline RAG

Les embeddings sont le pont entre le chunking (chapitre précédent) et la recherche sémantique. Voici comment ils s'intègrent dans le pipeline RAG complet :

Phase d'Indexation

Documents

Chunking

Embedding (par chunk)

Stockage Vector DB

Phase de Retrieval

Query utilisateur

Embedding (query)

Recherche Similarité

Top-K Chunks → LLM

Métriques de similarité : Cosine Similarity (le plus courant), Dot Product, Distance Euclidienne

Mesure de Similarité : Cosine Similarity

La Cosine Similarity mesure l'angle entre deux vecteurs. Valeur entre -1 et 1 : **plus la valeur est proche de 1, plus les textes sont sémantiquement similaires.**

$$\cos(A, B) = (A \cdot B) / (||A|| \times ||B||) = \sum(a_i \times b_i) / (\sqrt{\sum a_i^2} \times \sqrt{\sum b_i^2})$$

Exemple numérique (d=3)

Chunk A : "L'avocat va au tribunal" → emb_A = [0.716, 0.765, 0.604]

Chunk B : "Le juge rend son verdict" → emb_B = [0.698, 0.781, 0.612]

Chunk C : "La recette du guacamole" → emb_C = [-0.12, 0.890, -0.45]

$$\begin{aligned} \cos(A, B) &= (0.716 \times 0.698 + 0.765 \times 0.781 + 0.604 \times 0.612) / (1.210 \times 1.213) \\ &= (0.500 + 0.597 + 0.370) / 1.468 = 1.467 / 1.468 \approx 0.99 \quad \leftarrow \text{Très similaires !} \end{aligned}$$

$$\begin{aligned} \cos(A, C) &= (0.716 \times (-0.12) + 0.765 \times 0.890 + 0.604 \times (-0.45)) / (1.210 \times 1.008) \\ &= (-0.086 + 0.681 - 0.272) / 1.220 = 0.323 / 1.220 \approx 0.26 \quad \leftarrow \text{Peu similaires} \end{aligned}$$

Résumé du Chapitre

- Texte → Tokens → IDs → Vecteurs denses
- BPE / WordPiece / SentencePiece : 3 grandes familles de tokenisation
- Matrice d'Embedding E : lookup table $V \times d$
- Positional Encoding sinusoïdal : préserver l'ordre
- Self-Attention Q·K·V : contextualiser chaque token
- Pooling (CLS / Mean) : agréger en un seul vecteur
- Cosine Similarity : mesurer la proximité sémantique
- Pipeline RAG : Chunk → Embed → Store → Retrieve

Prochain chapitre : Vector Databases – Stocker et rechercher efficacement les embeddings.